MEMÓRIA DESCRITIVA

BookIT

Sistema distribuído de reservas com recomendação inteligente

Nome completo	Número de estudante
Yassin Latif	20240626
Yosvany Costa Nunes	20230853
Francisco Silva	20241293
Lueji Covilhã	20241725

Grupo	6
Curso	Licenciatura em Engenharia Informática
Unidade curricular	Projeto de Desenvolvimento de Software
Ano curricular / Semestre	3.º ano / 5.º semestre (2026/2027)
Docentes	Rui Pascoal e Miguel Boavida

1.ª entrega · Outubro de 2026

Resumo

O BookIT é um sistema de reservas de salas, equipamentos e lugares de estudo, pensado para uma universidade, onde muitas pessoas disputam os mesmos recursos. O utilizador diz o que precisa, por exemplo uma sala para seis pessoas, durante duas horas, com projetor, e o sistema devolve as melhores opções de recurso e horário, em vez de o obrigar a experimentar combinações à mão.

O sistema é distribuído. Um balanceador Nginx recebe os pedidos e reparte-os por duas ou mais instâncias da API de reservas. A API fala com duas instâncias de um serviço de IA e com uma base de dados PostgreSQL com um nó primário e uma réplica. Todos os componentes correm em contentores Docker, geridos com Docker Compose. A ideia é que a falha de uma instância da API, de uma instância da IA ou da réplica não pare o serviço e que nenhuma reserva confirmada se perca ou fique duplicada.

A consistência é garantida pela base de dados. Uma restrição de exclusão sobre intervalos de tempo recusa duas reservas sobrepostas no mesmo recurso, venha o pedido de que instância vier, e uma chave de idempotência evita criar a mesma reserva duas vezes quando um pedido é repetido depois de uma falha. A IA só sugere. Quem confirma é a base de dados, dentro de uma transação.

A recomendação usa um Problema de Satisfação de Restrições (CSP). As regras que não podem ser quebradas, como a ausência de sobreposição, a capacidade, o equipamento, o horário de funcionamento e as permissões, são restrições obrigatórias. As preferências do utilizador e a boa utilização dos espaços entram como custos a minimizar. A resolução usa backtracking com forward checking, as heurísticas MRV e de grau, o algoritmo AC-3 e branch and bound para devolver as k melhores soluções. O mesmo motor volta a distribuir as reservas afetadas quando um recurso fica indisponível.

Nesta primeira entrega o trabalho é de análise e desenho: requisitos, arquitetura, modelo de dados, modelo de faltas com cinco cenários de falha, protótipos de interface e planeamento. O protótipo será desenvolvido nas entregas seguintes e no fim comparamos o motor CSP com a estratégia simples de escolher o primeiro horário livre.

Contexto/ Introdução

1. Problema abordado

Reservar uma sala, um equipamento ou um lugar de estudo numa universidade obriga muitas vezes a ir tentando combinações de recurso, dia e hora até encontrar uma que sirva. As plataformas habituais mostram apenas o que está livre e deixam a escolha ao utilizador. Quando um recurso deixa de estar disponível, por avaria ou manutenção, as reservas afetadas têm de ser reorganizadas à mão.

Do ponto de vista técnico, o sistema tem de aguentar muitos pedidos ao mesmo tempo, incluindo pedidos para o mesmo recurso, e continuar a funcionar quando um componente falha, sem perder nem duplicar reservas confirmadas.

2. Motivação

Escolhemos este tema porque o problema é fácil de explicar e existe em qualquer universidade. Justifica uma arquitetura distribuída: há muitos utilizadores ao mesmo tempo, a procura tem picos, por exemplo na época de exames, e o serviço não deve depender de um único servidor. Permite também trabalhar um problema clássico dos sistemas distribuídos, a consistência perante concorrência e falhas. Por fim, escolher o melhor recurso e horário encaixa num tópico de Inteligência Artificial, os Problemas de Satisfação de Restrições.

3. Objetivos

- Criar, consultar e cancelar reservas sem sobreposições, mesmo com pedidos simultâneos.
- Manter o sistema a funcionar quando falha uma instância da API, uma instância da IA ou a réplica da base de dados.
- Não perder nem duplicar reservas confirmadas.
- Recomendar as melhores combinações de recurso e horário com um motor CSP e comparar o resultado com a escolha do primeiro horário livre.
- Reatribuir as reservas afetadas quando um recurso fica indisponível.
- Demonstrar com testes as garantias definidas no modelo de faltas.

Processo

4. Metodologia utilizada

Seguimos uma abordagem ágil, inspirada no Scrum, com sprints de duas semanas e uma reunião semanal para planear e fazer o ponto da situação. As tarefas ficam registadas no GitHub (Issues e Projects) e as alterações ao código entram por Pull Request, revistas por pelo menos um outro elemento do grupo.

5. Ferramentas utilizadas

GitHub para o repositório, as tarefas e a revisão de código. Docker e Docker Compose para o ambiente de execução. Graphviz para os diagramas e HTML e CSS para os mockups da interface.

6. Tecnologias utilizadas

Esta stack é uma proposta e ainda tem de ser validada pelos docentes.

- **Frontend:** React (aplicação web).
- **Backend:** Python com FastAPI, para a API de reservas e para o serviço de IA.
- **IA:** motor CSP em Python, feito por nós.
- **Base de dados:** PostgreSQL, com 1 primário e 1 réplica.
- **Balanceador:** Nginx.
- **Execução:** Docker e Docker Compose.

7. Estrutura da equipa

O Grupo 6 tem quatro estudantes: Yassin Latif, Yosvany Costa Nunes, Francisco Silva e Lueji Covilhã. O trabalho divide-se em quatro áreas: API e lógica de reservas, interface web, motor de IA, e base de dados com infraestrutura. Cada elemento fica responsável por uma área, mas todos participam na documentação, nos testes e na revisão do código dos colegas.

Resultados

8. Descrição da solução desenvolvida

Nesta fase o trabalho foi de análise e desenho. O utilizador faz login na aplicação web e pede, por exemplo, uma sala para seis pessoas, durante duas horas, com projetor. O pedido passa pelo Nginx, que o entrega a uma das instâncias da API de reservas. A API vai buscar à base de dados os recursos e as reservas existentes e envia-os ao serviço de IA, que devolve as três a cinco melhores opções (Figuras 1 e 4).

O utilizador escolhe uma opção e confirma. A API grava a reserva numa transação na base de dados. Se entretanto outra pessoa ficou com a sala, a base de dados recusa a gravação e o sistema recalcula as opções.

A arquitetura (Figura 1) tem um balanceador Nginx, duas ou mais instâncias da API de reservas, duas instâncias do serviço de IA e uma base de dados PostgreSQL com um primário e uma réplica, tudo em contentores Docker. As instâncias da API e da IA não guardam estado, por isso qualquer uma pode atender qualquer pedido.

O motor de IA trata a escolha como um CSP (Figura 5). A variável é o pedido de reserva e o domínio são os pares (recurso, intervalo) possíveis. As restrições obrigatórias são a ausência de sobreposição, a capacidade, o equipamento pedido, o horário de funcionamento e as permissões. As preferências de local e hora, o desperdício de capacidade, o uso equilibrado dos recursos e a fragmentação do calendário entram como custos a minimizar. A resolução usa backtracking com forward checking, MRV, AC-3 e branch and bound para devolver as k melhores soluções.

Fizemos ainda os casos de uso (Figura 2), o modelo de dados (Figura 3), os mockups da interface (Figuras 7 a 9) e o modelo de faltas (documento entregue à parte), com cinco cenários de falha para demonstrar com testes.

9. Funcionalidades principais

- **Utilizador:** fazer login, pesquisar a disponibilidade de recursos, pedir uma recomendação de recurso e horário, criar e cancelar reservas e consultar as suas reservas.
- **Gestor de recursos:** gerir recursos e equipamentos, marcar um recurso como indisponível e reatribuir as reservas afetadas, com ajuda da IA.
- **Administrador:** gerir utilizadores e permissões e consultar todas as reservas.
- Reservas sem sobreposições, mesmo com pedidos simultâneos.

10. Contributos relevantes

- Um modelo de faltas explícito, com as garantias e as limitações definidas e cenários de falha que vamos demonstrar com testes.
- A consistência das reservas assegurada pela própria base de dados, com uma restrição de exclusão sobre intervalos de tempo, independentemente de quantas instâncias da API existam.
- A aplicação de um CSP a um problema real de afetação de recursos, com o mesmo motor a recomendar e a reatribuir reservas.

Reflexão/ Conclusões

11. Lições aprendidas

Ao escrever o modelo de faltas percebemos que é preciso decidir cedo o que o sistema garante e o que não garante, e isso influenciou a arquitetura, por exemplo o número de instâncias por camada. Separar quem sugere (a IA) de quem confirma (a base de dados) torna mais simples garantir a consistência. Também ficou claro que convém começar por uma arquitetura simples e só acrescentar componentes quando forem necessários.

12. Limitações

Ainda não existe protótipo, por isso não há resultados experimentais nem medições. O Nginx é único e é um ponto único de falha: se falhar, o sistema pára. A promoção da réplica a primário é manual. O ambiente de demonstração corre numa só máquina, por isso só simulamos falhas de contentores e de rede. O modelo considera paragens de componentes e não comportamentos maliciosos. A meta de 2 segundos para a recomendação é um objetivo inicial, a ajustar com medições.

13. Trabalho futuro

Nas próximas entregas vamos implementar o protótipo (base de dados com réplica, API de reservas, motor CSP inicial e interface web), fazer os testes de falha e de concorrência e comparar o motor CSP com a escolha do primeiro horário livre. Mais tarde, se houver tempo, pode juntar-se um segundo balanceador para retirar o ponto único de falha e um modelo simples de Machine Learning para prever a procura por dia e hora.

Imagens

As figuras seguintes documentam o processo de trabalho, os conceitos, os protótipos e interfaces (mockups), os artefactos produzidos e os testes planeados. Como ainda não existe produto final, não incluímos imagens dele. Todas as imagens têm mais de 3000 px no maior lado e estão também no arquivo documental, na pasta 02_Imagens.

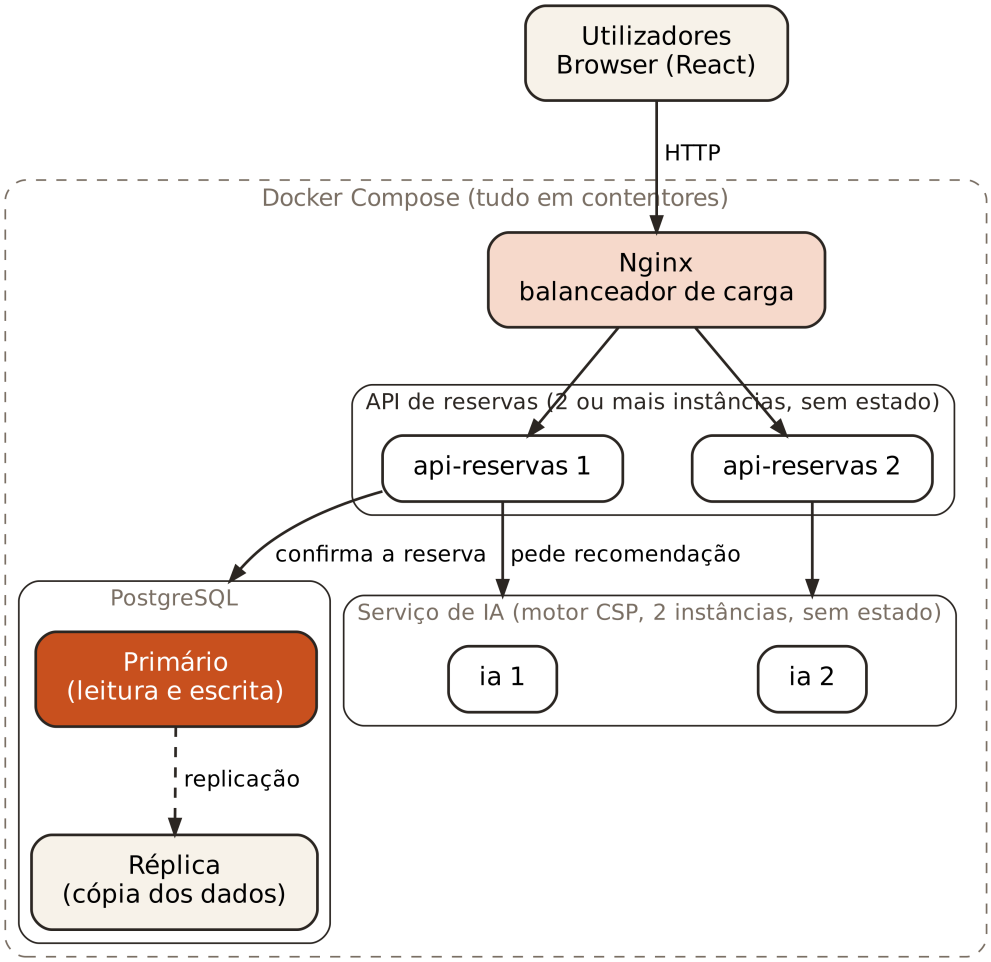


Figura 1. Arquitetura do sistema

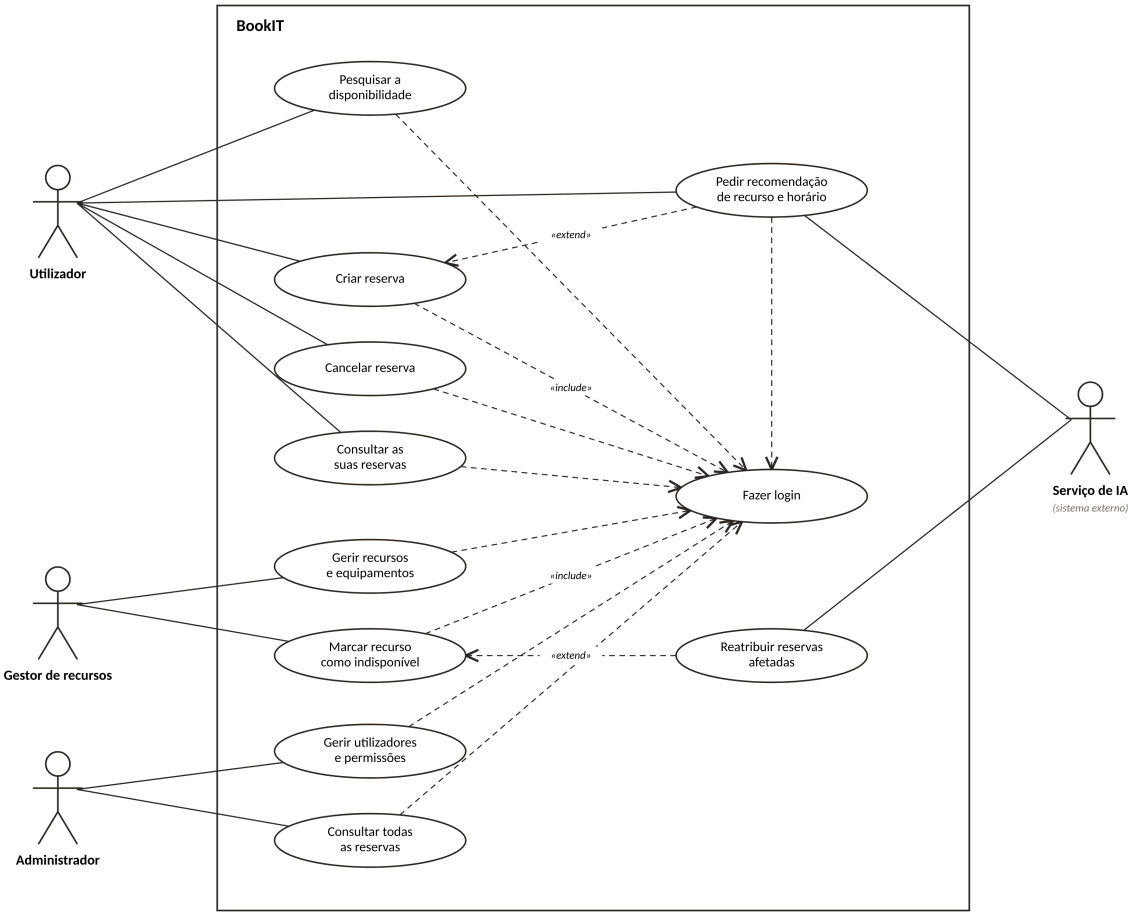


Figura 2. Diagrama de casos de uso

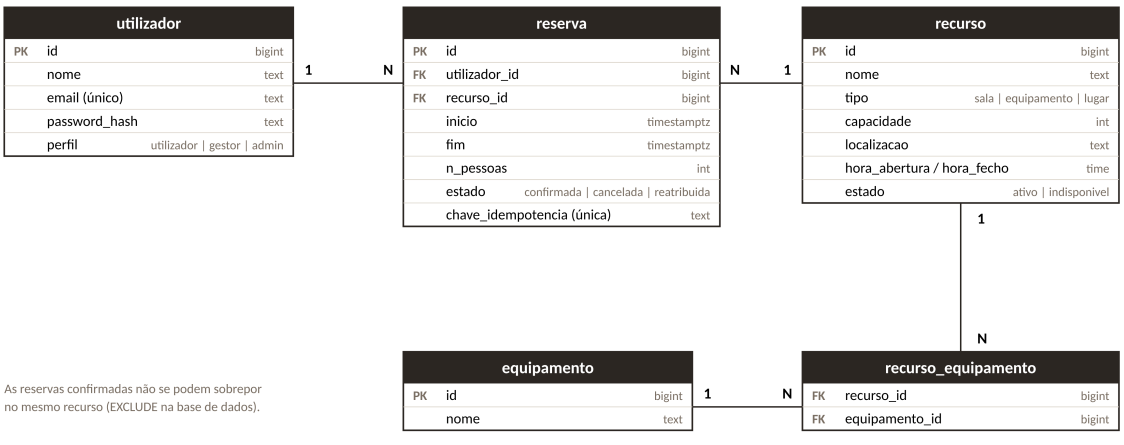


Figura 3. Modelo de dados

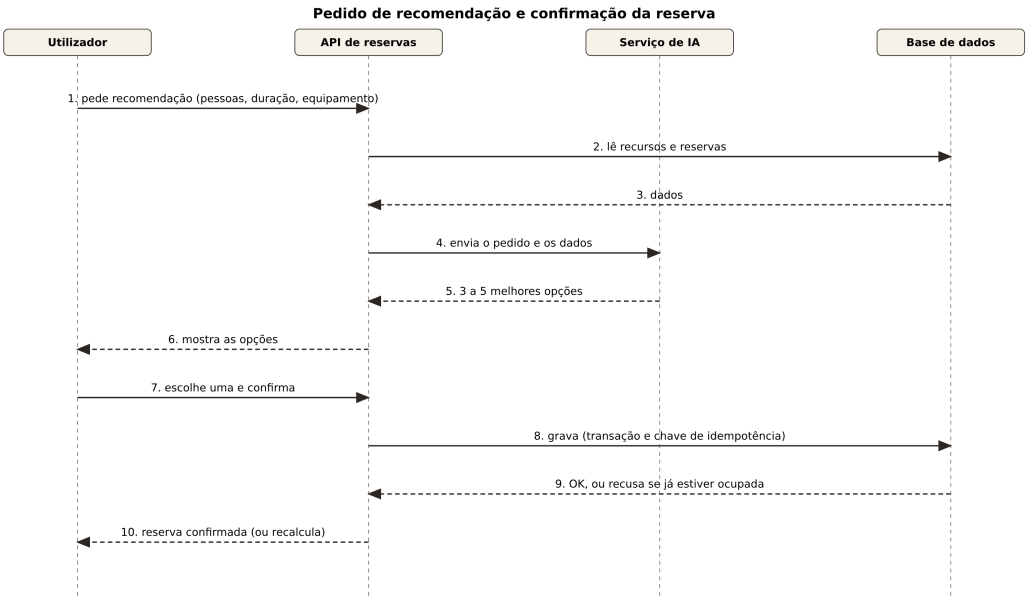


Figura 4. Sequência do pedido de recomendação e da confirmação da reserva

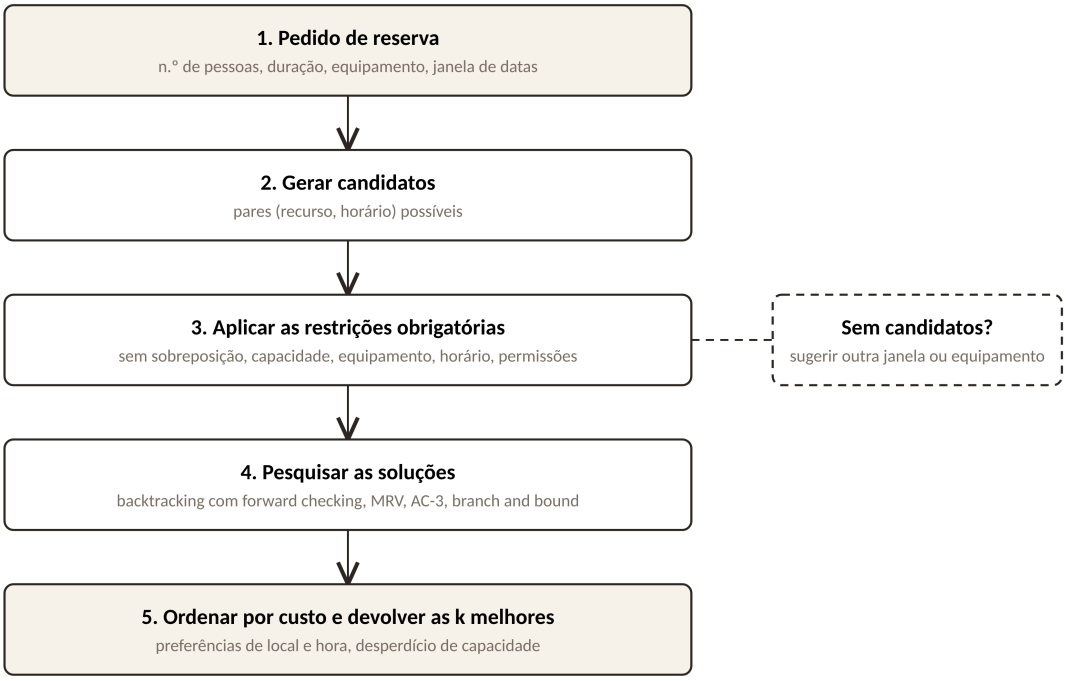


Figura 5. Fluxo do motor CSP

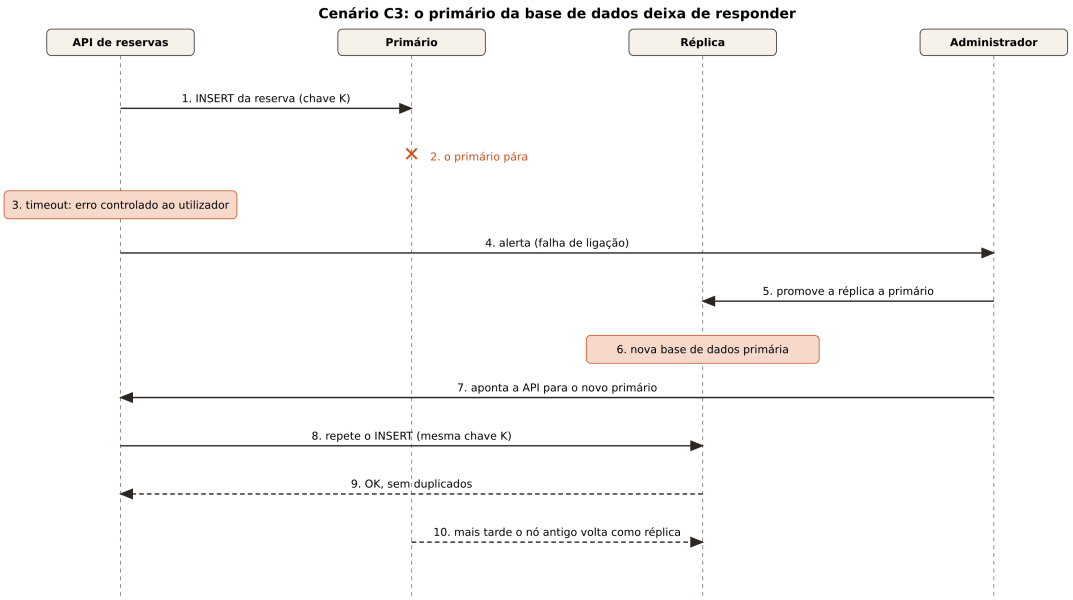


Figura 6. Cenário de teste C3: falha do primário da base de dados

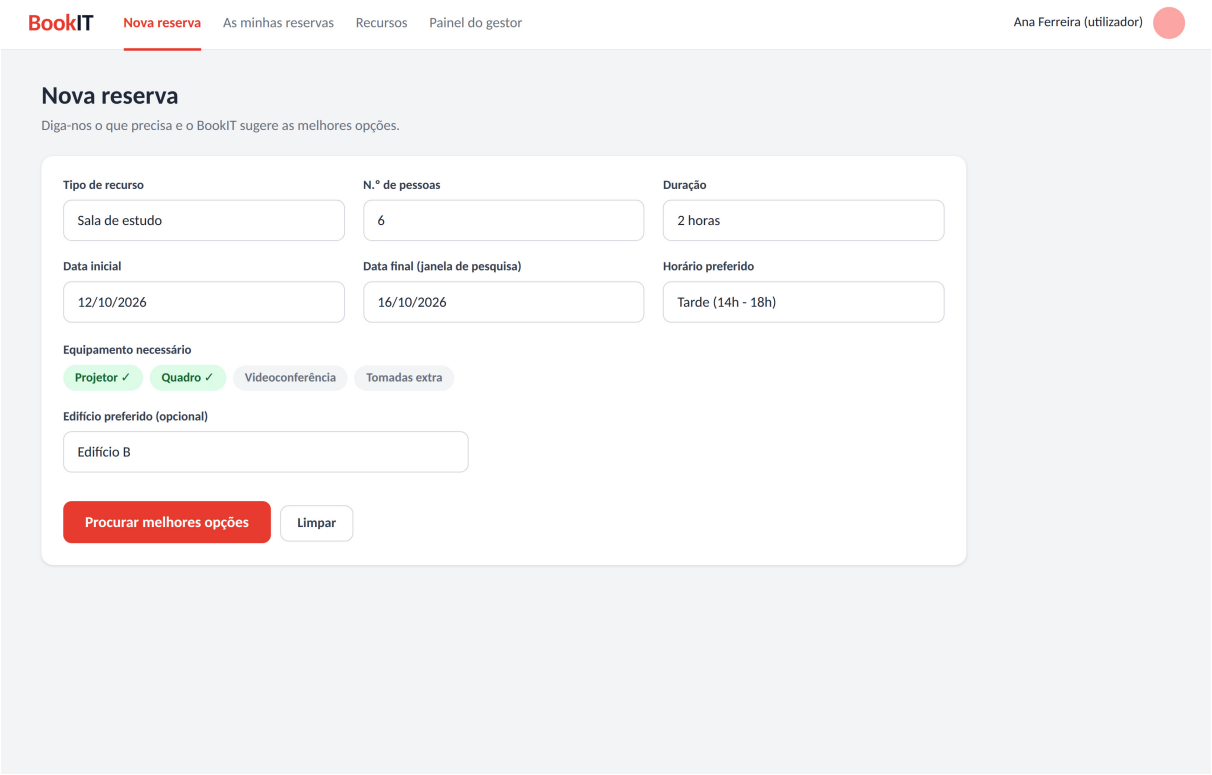


Figura 7. Mockup do ecrã Nova reserva

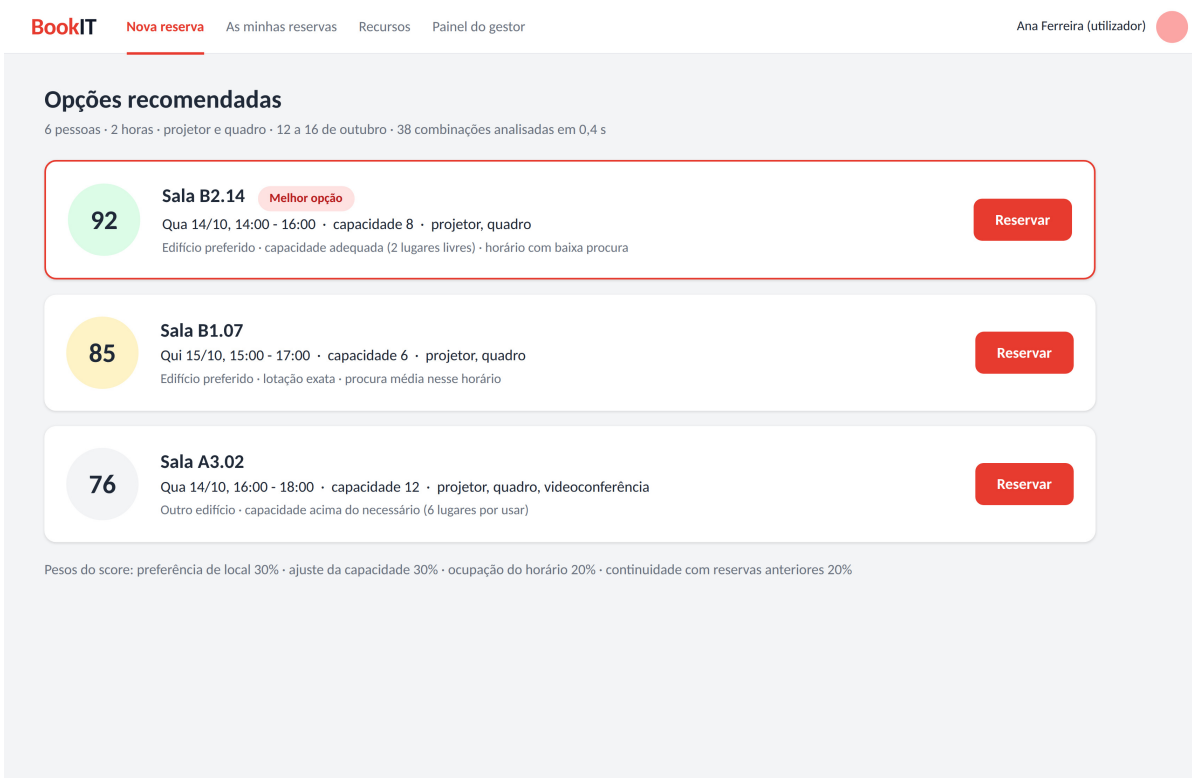


Figura 8. Mockup do ecrã com as opções recomendadas

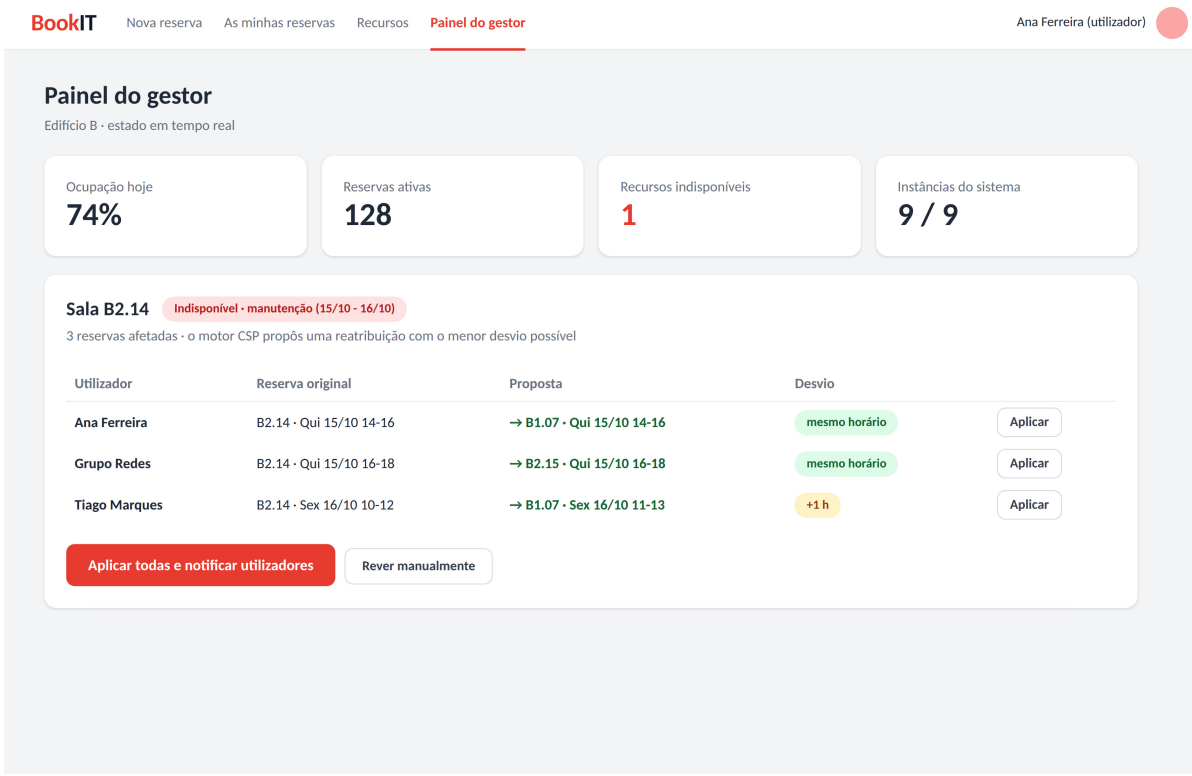


Figura 9. Mockup do painel do gestor, com a reatribuição de reservas

Modelo de faltas: cenários de falha a demonstrar

Uma falha de cada vez. Para cada cenário, o que falha, como se deteta, como o serviço se mantém, o impacto, a perda de dados e a recuperação.

	Componente que falha	Como se deteta	Como o serviço se mantém	Impacto e perda de dados	Recuperação e redundância
C1	Falha de uma instância da API	O Nginx deteta que a instância não responde	Os pedidos vão para as outras instâncias	Pedidos em curso falham e são repetidos. Sem perda de dados	O contentor reinicia e volta ao conjunto
C2	Falha de uma instância da IA	Timeout e health check	A API usa a outra instância. Sem IA, sugere o primeiro horário livre	Recomendação mais simples. Sem perda de dados	O contentor reinicia e volta a ser usado
C3	Falha do primário da base de dados	Falha de ligação e health check	A réplica tem uma cópia e passa a primário (promoção manual)	Escritas indisponíveis até à promoção. Sem perda de reservas confirmadas	O nó antigo volta como réplica e a replicação é reposta
C4	Falha da réplica	Estado da replicação no primário	O primário continua a funcionar, com um ajuste manual à replicação	Escritas em pausa até esse ajuste. Sem perda de dados	A réplica reinicia, volta a sincronizar e repõe-se a replicação síncrona
C5	Corte de rede entre a API e a base de dados	Timeout nas ligações	Erro controlado e nova tentativa	Reservas falham temporariamente. Sem duplicados (chave de idempotência)	Reconexão quando a rede volta

Figura 10. Cenários de falha a testar

Stack tecnológica proposta

Proposta, a validar com os docentes.

Frontend	React (aplicação web)
Backend	Python com FastAPI (API de reservas e serviço de IA)
IA	Motor CSP em Python, feito por nós
Base de dados	PostgreSQL, com 1 primário e 1 réplica
Balanceador	Nginx
Execução	Docker e Docker Compose
Código e tarefas	GitHub

Figura 11. Stack tecnológica proposta

Planeamento do projeto

Outubro a dezembro de 2026. Os losangos são as três entregas.

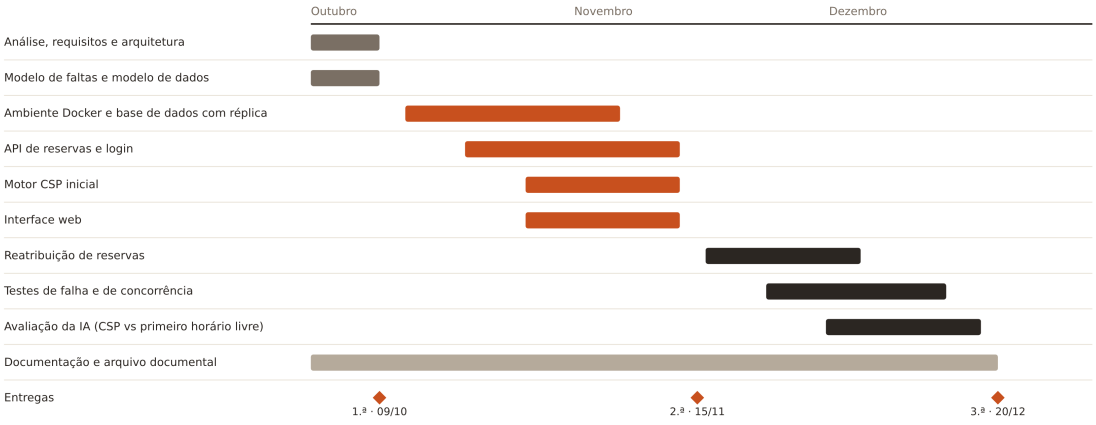


Figura 12. Planeamento do projeto, de outubro a dezembro de 2026

Ficheiros técnicos

- **Arquitetura do sistema:** Figura 1.
- **Diagramas UML:** casos de uso (Figura 2) e diagrama de sequência (Figura 4).

- **Estrutura da base de dados:** Figura 3 e ficheiro estrutura_base_dados.sql, na pasta 04_Documentacao_Tecnica do arquivo documental.
- **Capturas da aplicação:** ainda não existe aplicação. Os mockups estão nas Figuras 7 a 9.
- **Repositório GitHub:** ainda não existe. Será criado e disponibilizado na 2.ª entrega, com o protótipo.